

Table of Contents

- 1. Introduction and robot's initial overview
 - 1.1. Team Introduction
- 2. Robot's general logic and behavior
 - 2.1. Torsion and Strike Strategy
 - 2.2. State Machine and Maneuvers
- 3. Electronic hardware
 - 3.1. Main Microcontroller
 - 3.2. Sensors and measuring units
 - 3.2.1. IMU (Inertial Measurement Unit)
 - 3.2.2. Pixy v2 Camera
 - 3.2.3. Ultrasonic sensors
 - 3.2.4. Custom Ball Detection Board
- 4. Mechanical and structural hardware
 - 4.1. Differential Drive Configuration
 - 4.2. Roller (Active Dribbler)
 - 4.3. Chassis and Design

1. Introduction and robot's initial overview

1.1. Team Introduction

We are the Italian soccer Infrared team **millihertz**. Our team consists of two students, **Alessandro Campagna** and **Kristian Casalloni**, from the *Lorenzo Cobianchi Institute* in Verbania, collaborating with the *Elettra Robotics Lab*.

This documentation covers the design, hardware architecture, and strategic software implementation of our custom two-wheeled soccer robot built for national and international Robocup Junior robotic soccer matches.

Our team structure is organized as follows:

- **Alessandro Campagna & Kristian Casalloni:** Co-developed the full system integration, specializing in the Arduino Leonardo software layout, hardware assembly, Pixy v2 vision calibration, and the structural design of the two-wheeled differential drive chassis.

2. Robot's general logic and behavior

Unlike omnidirectional platforms, our robot utilizes a **two-wheeled differential drive kinematics system**. This mechanical choice requires a dedicated tactical behavior to optimize movement efficiency and minimize the time needed to strike.

2.1. Torsion and Strike Strategy

The core tactic of **millihertz** is engineered around an aggressive, high-speed interception loop divided into three main operational phases:

1. **Ball Tracking & Intake:** The robot uses its circular infrared array to detect the ball's position. It approaches the ball dynamically until the active **Roller** mechanism engages it, securing possession against the chassis.

```

2. void cerca_Palla() {
3.
4.     lettura_Palla();
5.     verifica_posizione();
6.     leggi_imu();
7.
8.     if((val_imu<225 && val_imu>135) && (digitalRead(IR_VICINO)!=0))
9.     {
10.         m_rullo();
11.         m_avanti_soft();
12.         if(digitalRead(TOCC0)==1){
13.
14.             if(analogRead(US_DX)>analogRead(US_SX)){
15.
16.                 while(val_imu>45 && val_imu<315 &&
17.                 digitalRead(TOCC0)==1){
18.
19.                     m_ruota_dx();
20.                     leggi_imu();
21.                 }
22.
23.             } else {

```

```

24.
25.         while(val_imu>45 && val_imu<315 &&
digitalRead(TOCC0)==1){
26.
27.             m_ruota_sx();
28.             leggi_imu();
29.
30.         }
31.     }
32.
33.     stacca_muro();
34.
35.     }
36.     //autogol();
37. }
38.     if (DIREZIONE <= 6 && DIREZIONE >=4) { //PALLA SITUATA
DAVANTI ALLA SCHEDA PALLA
39.         m_rullo();
40.         m_avanti();
41.     }
42.     else if (DIREZIONE > 6 && DIREZIONE < 12) { //LA PALLA SI
TROVA A DESTRA
43.         m_rullo_fermo();
44.         if (DIREZIONE < 8 && DIREZIONE > 5) {
45.             m_ruota_dxsoft();
46.         } else m_ruota_dx();
47.     }
48.     else if (DIREZIONE > 12 || DIREZIONE < 5) { //LA PALLA SI
TROVA A SINISTRA
49.         m_rullo_fermo();
50.         if (DIREZIONE < 4 && DIREZIONE > 1) {
51.             m_ruota_sxsoft();
52.         } else m_ruota_sx();
53.     }
54.     else if (DIREZIONE == 0) { //NON RILEVA LA PALLA
55.         ritorna_porta();
56.     }
57. }

```

58. Torsion Maneuver (Pivot Rotation): Once the ball is trapped by the roller, the robot stops its forward movement and performs a precise, zero-radius spin (pivot turn) on its central axis. The direction of the spin is determined by the last known position of the opponent's goal or the absolute heading calculated by the onboard IMU.

```

59.     void punta_porta() {

```

```

60.
61.     leggi_imu(); //serve ad aggiornare la imu
62.     pixy.ccc.getBlocks();
63.
64.     /* DEBUG: vedi cosa leggono i sensori
65.     Serial.print("IMU: "); Serial.print(val_imu);
66.     Serial.print(" | Pixy: ");
67.     Serial.println(pixy.ccc.numBlocks);
68.     */
69.     if(analogRead(US_FRONTALE)<=25){// MI STACCO DAL MURO
70.
71.         stacca_muro();
72.         return; //esco dalla funzione
73.     }
74.
75.     if (pixy.ccc.numBlocks == 1 || (val_imu > 330 &&
76.     val_imu<20) ) {// SE VEDO LA PORTA AVVERSARIA ATTACCO
77.
78.         attacco();
79.
80.     } else if ( analogRead(US_SX)>analogRead(US_DX) ) {// MI
81.     RIPOSIZIONO VERSO LA PORTA SE MI SONO SPOSTATO val_imu >= 30 &&
82.     val_imu <= 180
83.
84.     while(pixy.ccc.numBlocks == 0 && (val_imu < 330 &&
85.     val_imu > 20) && digitalRead(TOCC0) == 1 ){
86.
87.         m_gira_sx_soft();
88.         pixy.ccc.getBlocks();
89.         leggi_imu();
90.
91.         if(analogRead(US_FRONTALE)<=25){// MI STACCO DAL MURO
92.
93.             stacca_muro();
94.             return; //esco dalla funzione
95.         }
96.     }
97.     } else {
98.
99.         while(pixy.ccc.numBlocks == 0 && (val_imu <330 &&
100.     val_imu >20 ) && digitalRead(TOCC0) == 1 ){
101.
102.             m_gira_dx_soft();
103.             pixy.ccc.getBlocks();
104.             leggi_imu();
105.
106.             if(analogRead(US_FRONTALE)<=25){// MI STACCO DAL MURO

```

```

102.         stacca_muro();
103.         return; //esco dalla funzione
104.     }
105. }
106. }

```

107. **Alignment & Strike:** As the robot rotates, the **Pixy v2 smart camera** continuously scans for the opponent's goal color signature. The moment the goal enters the center of the camera's field of view, the rotation stops instantly, and the robot drives straight forward to score.

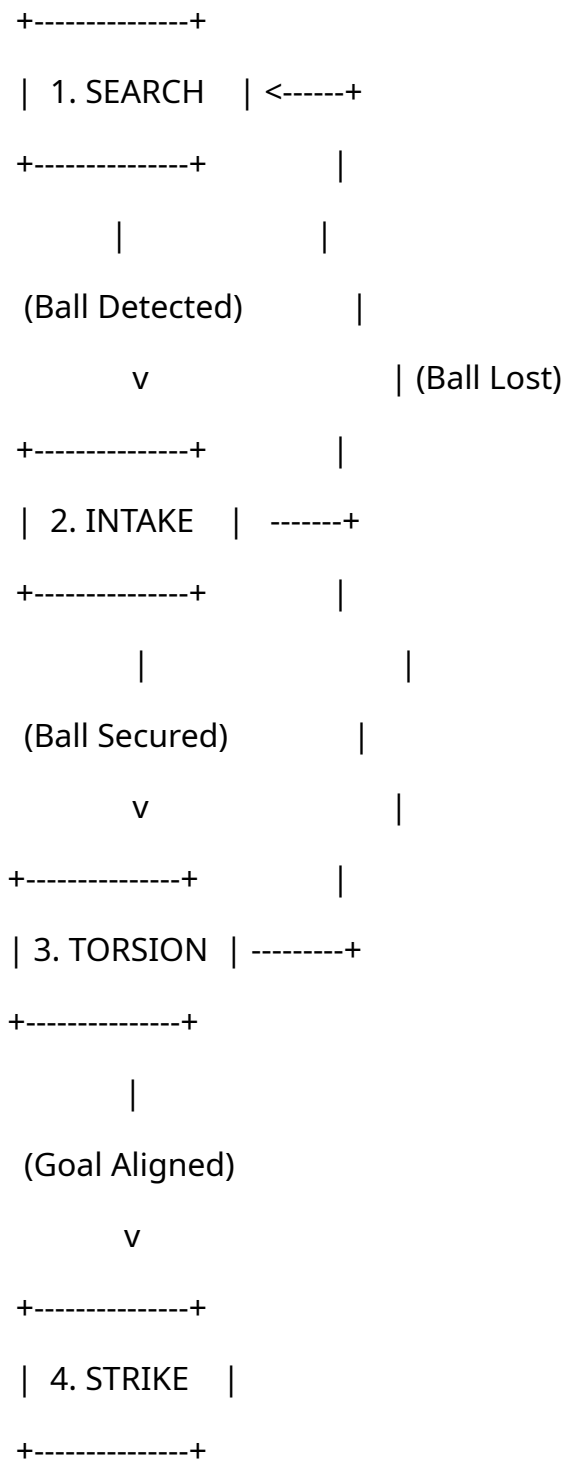
```

108.     void attacco() {
109.
110.         pixy.ccc.getBlocks();
111.         leggi_imu();
112.         while (pixy.ccc.numBlocks == 1 || (val_imu>330 ||
val_imu<20) && digitalRead(TOCC0) == 1 ) { //270 90
113.
114.             pixy.ccc.getBlocks();
115.             leggi_imu();
116.             m_avanti();
117.
118.         }
119.     }

```

2.2. State Machine and Maneuvers

The software structure inside the main MCU processes this strategy using a **Finite State Machine (FSM)**. Below is a conceptual breakdown of the algorithm's states:



- **State 1: Search** -> The robot spins or drifts until the *Ball Detection Board* receives a strong infrared signal.
- **State 2: Intake** -> The primary motors drive forward while the *Roller motor* spins at high speed to swallow and control the ball.

- **State 3: Torsion** -> Left and right motors rotate in opposite directions at equal speeds ($V_{\text{Left}} = -V_{\text{Right}}$). The robot rotates on its geometric center, ensuring the ball stays pinned inside the roller.
- **State 4: Strike** -> Both motors accelerate forward ($V_{\text{Left}} = V_{\text{Right}}$) to push the ball straight into the targeted goal area.

3. Electronic hardware

The electronic framework of **millihertz** is designed to process high-frequency sensor readings and translate them into real-time motor corrections.

3.1. Main Microcontroller

- **Arduino Leonardo Board:** The brain of the robot. This board features the ATmega32u4 8-bit microcontroller. It was chosen for its built-in USB communication (excellent for real-time debugging and field calibrations), sufficient analog/digital I/O lines, and its reliable capability to handle fast hardware interrupts coming from the sensor modules.

3.2. Sensors and measuring units

3.2.1. IMU (Inertial Measurement Unit)

Equipped with a combination of a digital **gyroscope and accelerometer**. The IMU provides the robot with an absolute spatial orientation vector. It acts as a digital compass to counteract drift during high-speed collisions and ensures the robot maintains a strict spatial awareness relative to the field coordinates, even before the camera locks onto the goal.

3.2.2. Pixy v2 Camera

A dedicated, high-speed **smart vision sensor** connected via serial link to the Arduino Leonardo. The Pixy v2 filters and recognizes the color signatures of the opponent's goal frames autonomously at 60 frames per second, offloading heavy image processing tasks from the central MCU. It returns specific coordinate blocks indicating whether the goal is centered, to the left, or to the right of the robot's heading.

3.2.3. Ultrasonic sensors

Mounted on the outer perimeter of the chassis, these sensors use high-frequency sound echoes to compute the precise distance to the field walls. They

prevent the robot from getting stuck in corners and help center the robot dynamically when returning to defend.

3.2.4. Custom Ball Detection Board

An advanced, circular custom-designed PCB populated with **16 Infrared (IR) sensors** arranged in a perfect 360° array. This board intercepts the pulsed infrared light emitted by the official tournament ball. By comparing the analog intensity values across the 16 distinct sectors, the algorithm calculates the exact vector (Angle and Distance) of the ball relative to the robot's face.

4. Mechanical and structural hardware

4.1. Differential Drive Configuration

Unlike omnidirectional robots that sacrifice pushing force for side-way movements, the **millihertz** platform features a highly reliable **two-wheel differential configuration** combined with low-friction passive gliders. This mechanical setup yields significant torque and acceleration advantages when driving forward to strike or pushing against opponent structures.

4.2. Roller (Active Dribbler)

The front intake mechanism features a custom-built **Roller** covered with high-grip material, powered by a dedicated motor. When active, it generates a backspin on the ball surface. This mechanical backspin forces the ball to stay safely nested inside the robot's mechanical jaw, allowing the robot to execute high-speed spin rotations (the *Torsion maneuver*) without letting the ball slip away due to centrifugal forces.

4.3. Chassis and Design

The mechanical structure is built upon a 3D-printed impact-resistant green PLA chassis layout designed for lightness, structural rigidity, and easy access to hardware layers.

The vertical structural frame holds the **Pixy v2 camera** at an optimal elevated angle to prevent other robots from obstructing its field of view, ensuring a continuous line of sight to the opponent's goals.

